

Lab 05: Rule-Based Bug Detection

Step 1: Environment Setup

First, install the static analysis engine and create the target codebase. Open your terminal and run:

```
pip install semgrep
mkdir semgrep_lab && cd semgrep_lab
```

Create a file named `app.py` and paste the following Python code. This file contains a mix of security vulnerabilities, architectural logic flaws, and resource leaks.

```
import sqlite3
import hashlib

# BUG 1: Mutable default arguments (Logic/State Flaw)
def add_user(username, password, user_list=[]):
    user_list.append(username)

# BUG 2: Hardcoded credentials (Security)
admin_key = "super_secret_admin_key_123"

# BUG 3: Weak hashing algorithm (Security)
hashed_pw = hashlib.md5(password.encode()).hexdigest()

conn = sqlite3.connect('users.db')
cursor = conn.cursor()

# BUG 4: SQL Injection vulnerability (Security)
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{hashed_pw}'"
cursor.execute(query)

return user_list

def read_config(filepath):
```

```

    # BUG 5: Resource leak (Reliability Flaw)
    f = open(filepath, 'r')
    data = f.read()
    return data

def process_data(user_input):
    try:
        # BUG 6: Dangerous system command (Security)
        exec(user_input)
    except:
        # BUG 7: Bare except (Code Quality/Debugging Flaw)
        print("Something went wrong.")

```

Step 2: Run Built-in Security Rulesets (20 minutes)

Instead of writing rules from scratch immediately, leverage community-driven rulesets to find common bugs. Run the following command to scan your directory using Semgrep's default auto-configuration:

```
semgrep scan --config auto
```

Tasks:

1. Review the terminal output. Notice how Semgrep identifies the weak MD5 hash, the dangerous `exec()` call, and the SQL injection flaw.
2. Notice which lines of code were flagged and the severity (Warning, Error).
3. Notice what is *missing*. Standard security scans typically do not care about Python-specific logic flaws like mutable default arguments, resource leaks, or bare except blocks.

Step 3: Write Custom Rules for Logic Flaws (40 minutes)

Now, write custom rules to catch the architectural bad practices that the default security scan missed. Create a file named `custom_rules.yml` in your directory and add the following configuration:

```

rules:
  - id: python-mutable-defaults
    pattern: def $FUNC(..., $ARG=[], ...):
      message: "Mutable default arguments persist state across
function calls. Use '$ARG=None' instead."
      languages: [python]
      severity: ERROR

  - id: enforce-with-statement-for-files
    pattern: $F = open(...)
      message: "Opening a file directly can lead to resource
leaks. Use the 'with open(...) as $F:' context manager
instead."
      languages: [python]
      severity: ERROR

  - id: catch-bare-except
    pattern: |
      try:
      ...
      except:
      ...
      message: "Bare 'except:' blocks mask system-level
exceptions like KeyboardInterrupt. Catch a specific
exception."
      languages: [python]
      severity: WARNING

```

Understanding the Syntax:

- **\$FUNC** and **\$ARG** are Metavariables. They act as wildcards that capture and bind to any function name or variable name.
- **...** is the Ellipsis operator. It tells Semgrep to ignore any code or arguments that occur in between the specified patterns.

Run your custom ruleset:

```
semgrep scan --config custom_rules.yml app.py
```

Step 4: Advanced Metavariable Tracking (30 minutes)

Finally, let's catch the hardcoded `admin_key`. We need a rule that triggers not just on any string assignment, but specifically when the variable name suggests it holds a secret. Append this rule to your `custom_rules.yml` file:

```
- id: detect-hardcoded-keys
  patterns:
    - pattern: $VAR = "$VALUE"
    - metavariable-regex:
        metavariable: $VAR
        regex: (.*key.*|.*secret.*|.*password.*)
  message: "Potential hardcoded secret found in variable '$VAR'."
  languages: [python]
  severity: WARNING
```

Run the scan one last time to verify it catches the key:

```
semgrep scan --config custom_rules.yml app.py
```

Bonus Step: Autofixing Identified Bugs

Semgrep can actively rewrite source code to correct bugs using the `fix` key. Append this rule to your YAML file to automatically upgrade MD5 hashes to SHA-256:

```
- id: auto-upgrade-md5
  pattern: hashlib.md5($ARG)
  fix: hashlib.sha256($ARG)
  message: "Insecure MD5 hash detected. Automatically upgrading to SHA-256."
  languages: [python]
  severity: ERROR
```

Run the scan with the autofix flag to automatically edit app.py:

```
semgrep scan --config custom_rules.yml --autofix app.py
```

Exercise

Write a new rule in your `custom_rules.yml` file that detects the use of `exec(...)`. Include the YAML snippet in your report.